

CSCE 641 COMPUTER GRAPHICS · MODULE 03 · LECTURE NOTES

Transformations: Model, View, Projection

Deriving every matrix in the vertex shader.

MODULE OUTCOMES

- Derive translation, rotation, and scale matrices in homogeneous form.
- Explain why matrix order matters and predict the result of swapping two.
- Construct a view matrix from a camera position and orientation.
- Derive the perspective projection matrix and explain each entry.
- Explain depth precision, why it is non-linear, and how to improve it.

1 Model: placing an object

The model matrix takes a vertex from the space the mesh was authored in to a shared world space. It is almost always a product of translation, rotation, and scale, and the order you multiply them in determines what the object does.

1.1 The three primitives

Translation $T(t_x, t_y, t_z)$	Scale $S(s_x, s_y, s_z)$	Rotation about Z, angle a
$\begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} \cos a & -\sin a & 0 & 0 \\ \sin a & \cos a & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

Translation lives entirely in the fourth column, which is why it only reaches vertices with $w = 1$. Scale is diagonal. Rotation is orthonormal with determinant $+1$ — a determinant of -1 would be a reflection, which flips winding order and makes your meshes render inside out.

1.2 Order

THE TWO ORDERS MEAN DIFFERENT THINGS

T · R — rotation applied first, in the object's own frame, then the result is translated. The object spins on its own axis and then moves. This is what you almost always want.

R · T — translation first, then the whole displaced object is rotated about the world origin. The object orbits. Useful for exactly that, and a bug everywhere else.

The conventional model matrix is $T \cdot R \cdot S$: scale the object in its own frame, orient it, then place it. Reading right to left gives the sequence of operations applied to the object; reading left to right gives the sequence of frame changes. Both readings are valid, they are inverses of each other, and mixing them within a single debugging session is a reliable way to waste an afternoon.

2 View: the camera that is not there

The hardware renders whatever lands in the canonical view volume, viewed from the origin looking down $-Z$. It has no notion of a camera anywhere else. So rather than moving a camera to the scene, we transform the scene into the camera's frame.

$$V = (\text{camera's world transform})^{-1}$$

If the camera's world transform is a rotation R followed by a translation to position $\mathbf{e}$, then $V = (T(\mathbf{e}) \cdot R)^{-1} = R^{-1} \cdot T(-\mathbf{e}) = R^T \cdot T(-\mathbf{e})$, using orthonormality from Module 02. That identity is what makes the view matrix essentially free to construct — no general inverse is ever computed.

2.1 look-at

Given an eye position, a target, and an approximate up vector, build the camera's orthonormal basis and write it transposed:

```
mat4 look_at(vec3 eye, vec3 target, vec3 up) {
    vec3 f = normalize(eye - target); // camera +Z points BACKWARD
    vec3 r = normalize(cross(up, f)); // camera +X, right
    vec3 u = cross(f, r);           // camera +Y, true up
                                   // (orthogonalises a sloppy `up`)

    return mat4(
        r.x, r.y, r.z, -dot(r, eye), // basis vectors go in the ROWS:
        u.x, u.y, u.z, -dot(u, eye), // that is the transpose,
        f.x, f.y, f.z, -dot(f, eye), // which is the inverse.
        0, 0, 0, 1);
}
```

TWO THINGS THAT BITE HERE

f points backward. OpenGL's camera looks down $-Z$, so the camera's own $+Z$ axis points behind it. Get this sign wrong and you render the world behind the camera — usually an empty screen, occasionally a confusing mirror image.

The supplied 'up' need not be exact. It only has to be non-parallel to f ; the double cross product orthogonalises it. But if it is parallel — look straight up with $up = (0,1,0)$ — the first cross product degenerates and the view matrix fills with NaN. This is the classic 'camera explodes at the poles' bug.

3 Projection

Projection has two jobs, and the second is less obvious than the first.

- **Foreshortening.** Distant objects must appear smaller, which requires dividing by depth.
- **Normalising the clip volume.** Map the frustum — a truncated pyramid — onto the cube $[-1,1]^3$, so that clipping becomes six comparisons against constants instead of six arbitrary plane tests. This is the job that actually justifies the stage.

3.1 Deriving the perspective matrix

Place a projection plane at distance n in front of the eye. By similar triangles, a point at view-space (x, y, z) with z negative projects to:

$$x' = n \cdot x / (-z) \quad y' = n \cdot y / (-z)$$

Division is not a linear operation, so no 4×4 matrix can perform it. But homogeneous coordinates already *end* in a division — the perspective divide by w . So we do not need the matrix to divide; we need it to arrange the right numerator and denominator. Setting the last row to $(0, 0, -1, 0)$ makes $w_{clip} = -z_{view}$, and the hardware divide does the rest.

The remaining freedom is row 3, which maps depth into $[-1, 1]$. Writing it as $z_{clip} = Az + B$ and requiring $z_{ndc} = -1$ at $z = -n$ and $+1$ at $z = -f$ gives:

$$A = -(f+n)/(f-n) \quad B = -2fn/(f-n)$$

```
mat4 perspective(float fovy, float aspect, float n, float f) {
    float t = 1.0f / tanf(fovy * 0.5f);
    return mat4(
        t/aspect, 0, 0, 0,
        0, t, 0, 0,
        0, 0, -(f+n)/(f-n), -2*f*n/(f-n),
        0, 0, -1, 0);
}
```

Verify it by hand once. Put $z = -n$ into row 3: $z_{clip} = n(f+n)/(f-n) - 2fn/(f-n) = n(n-f)/(f-n) = -n$. And $w_{clip} = n$. So $z_{ndc} = -1$. The far plane checks out the same way. Doing this once removes all mystery

from the matrix.

3.2 Orthographic

An orthographic projection has no divide — the last row stays $(0,0,0,1)$, so w remains 1. It is a scale and offset mapping a box onto the cube. Parallel lines stay parallel, depth precision is uniform, and there is no foreshortening. Beyond CAD and 2D overlays, the important use is **directional-light shadow maps** in Module 10: the sun is infinitely far away, so its 'camera' is orthographic.

4 Depth precision and z-fighting

Having divided by w , the stored depth is a function of $1/z$ rather than z . Depth buffer values are therefore distributed hyperbolically through the scene: densely near the camera, sparsely far away. The distribution depends almost entirely on the ratio f/n .

Near	Far	f/n	Fraction of depth range used by the nearest 10% of the view distance
0.01	1000	100,000	~91%
0.1	1000	10,000	~90%
1.0	1000	1,000	~82%
1.0	100	100	~53%

Indicative figures for a standard $[-1,1]$ depth mapping. The qualitative point is what matters: the near plane dominates, and a small f/n ratio is worth far more than a large depth buffer.

THE COUNTERINTUITIVE PART

Everyone's instinct on seeing z-fighting is to pull the *far* plane in. It barely helps.

Pushing the *near* plane out helps enormously. Going from $n = 0.01$ to $n = 0.1$ is a tenfold improvement in the f/n ratio for the cost of clipping geometry within 10cm of the camera — almost always an acceptable trade.

4.1 Reversed-Z

A near-free improvement. Map the near plane to 1 and the far plane to 0, use a floating-point depth buffer, and set the depth test to GREATER. Floating-point numbers are dense near zero; $1/z$ is also dense near... the far plane, once reversed. The two non-uniformities cancel almost exactly, and depth precision becomes nearly uniform across the whole range.

- Use a 32-bit float depth buffer (`GL_DEPTH_COMPONENT32F`).
- Set `glClipControl(GL_LOWER_LEFT, GL_ZERO_TO_ONE)`.
- Swap near and far in the projection matrix.
- Clear depth to 0 and set `glDepthFunc(GL_GREATER)`.
- Remember that any code reading depth — shadow maps, reconstruction from depth, SSAO — must be updated to match.

5 The chain, at the right frequency

Three matrices, built at three different rates. Building any of them more often than its natural frequency is a common and entirely avoidable cost.

Matrix	Rebuild when	Typical frequency
Projection P	Field of view, aspect ratio, or clip planes change	On window resize. Often once per run.
View V	The camera moves or turns	Once per frame.
Model M	The object moves	Once per object per frame, at most.
Normal matrix	M changes	With M. Never per vertex, never per fragment.

```
// Per frame
mat4 V = look_at(eye, target, up);

// Per object
mat4 M = T(position) * R(rotation) * S(scale);
mat4 MVP = P * V * M;
mat3 Nrm = transpose(inverse(mat3(M)));

// Per vertex, on the GPU
gl_Position = MVP * vec4(aPos, 1.0);
```

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
GAMES101 Lectures 03–04 — Transformation, Viewing	The clearest derivation of the perspective matrix available free, including the similar-triangles argument and the squish-then-ortho view.
Scratchapixel — The Perspective and Orthographic Projection Matrix	Full written derivation with every entry justified. The reference to reach for when a step in the derivation does not land.
LearnOpenGL — Coordinate Systems	The API mechanics: where each matrix goes and how to upload it.
NVIDIA — Depth Precision Visualized	The definitive treatment of depth precision and reversed-Z, with diagrams that make the 1/z distribution obvious.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement `translate`, `scale`, `rotate_x/y/z`, `look_at`, `perspective`, and `orthographic`. Unit-test each by transforming known points and checking the results by hand.
- 2 Verify the perspective matrix numerically: transform a point at $z = -n$ and one at $z = -f$, divide by w , and assert that `z_ndc` is -1 and $+1$ respectively.
- 3 Build the same scene with $T \cdot R$ and with $R \cdot T$ and render both. Capture the two images and write one sentence explaining the difference to someone who has not taken this course.
- 4 Plot depth-buffer value against world distance for f/n ratios of 100, 1,000, and 100,000. Then implement reversed-Z and plot it on the same axes.
- 5 Deliberately construct the degenerate look-at: camera looking straight down with $up = (0,1,0)$. Observe the NaN, then fix it, and write down how you would detect this automatically.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why does the projection matrix map the frustum to a cube, when foreshortening alone would not require that?
- 2 What specifically does the entry -1 in the last row of the perspective matrix accomplish?
- 3 The view matrix is described as 'the inverse of the camera transform'. Why is it cheap to compute despite being an inverse?
- 4 Explain why depth precision depends far more on the near plane than on the far plane.
- 5 How does reversed-Z recover precision? What two non-uniformities is it playing against each other?
- 6 State in which order you would multiply T , R , and S for a conventional model matrix, and explain what each of the other orders would do.